HirePro / Neo IIT Internship 2025

Compiled Practice Questions with Answers & Explanations

Sections: Aptitude · Data Structures · Operating Systems
C Programming · Digital Circuits · VLSI · Signals & Systems · Microprocessors

Compiled July 28, 2026

Table of Contents

Aptitude — Problem Solving

1. Glucose solution dilution
2. Shopkeeper markup & discount
3. Factory production rate
4. Two students on circular track

Aptitude — Puzzles

5. HOUSE : JQWUG :: PICTURE : ?
6. POCKET : PTOECK :: SILVER : ?
7. Star-operation word coding
8. GIFT / MUST / TRUE coding
9. Ram Mandir text – priority queue rearrangement

Data Structures Basics

10. Array vs Stack – correct difference
11. Queue-via-single-stack – time complexity
12. In-order predecessor of root (binary tree)
13. Bucket sort – true statements

Operating System Fundamentals

14. Priority scheduling algorithm – true statements
15. Best Fit allocation technique
16. I/O controller – OS function

C Programming

17. Bitwise shift/mask program output
18. printf format-mismatch program output
19. Array pointer error-finding (I)
20. Pointer-to-pointer error-finding (II)
21. NULL pointer assignment program

Digital Circuits & VLSI

22. 4-bit R-2R ladder DAC
23. Weighted-resistor DAC
24. 4×1 MUX – realized function
25. T flip-flop – input frequency
26. Excess-3 code – best use case
27. Testability – definition
28. PMOS – triode region condition
29. Logic families – characteristic matching
30. Boolean expression simplification (5-variable)
31. JK flip-flop counter – J1 & K1 equations
32. VLSI design methodology selection

Signals & Systems

33. Transfer function – causality & stability

Microprocessors

34. 8085 program – register & flag status

Aptitude — Problem Solving

Q1 *Aptitude / Problem Solving*

A 40% glucose (C6H12O6) solution is first diluted with water to make it a 32% solution (assume volumes add). This 32% solution is then mixed with a 20% glucose solution to obtain 50 L of a 26% glucose solution. How many liters of the original 40% solution were used?

16 L

20 L ✓

18 L

24 L

Step 1: Find how much 32% solution was used in the final mix.
0.32x + 0.20(50−x) = 0.26(50) ⇒ 0.12x = 3 ⇒ x = 25 L (32% solution)
Step 2: Find how much 40% solution made that 25 L of 32% solution.
0.40y = 0.32(25) ⇒ y = 8/0.40 = **20 L**

Q2 *Aptitude / Problem Solving*

A shopkeeper marks an item 40% above the cost price. He allows a discount of 10% on the marked price and still earns a profit of ₹126. What is the cost price of the item?

₹484.62 ✓

₹472.68

₹512.26

₹562.88

CP = x ⇒ Marked Price = 1.4x ⇒ SP after 10% discount = 1.4x × 0.9 = 1.26x
Profit: 1.26x − x = 0.26x = 126 ⇒ x = 126 / 0.26 = **₹484.62**

Q3 *Aptitude / Problem Solving*

A factory produces 400 units in 10 hours. How many units will it produce in 18 hours?

900

800

720 ✓

600

(400 / 10) × 18 = 40 × 18 = **720**

Q4 *Aptitude / Problem Solving*

Two students start together from the same point on a circular jogging track of length 900 m and run in the same direction. Their speeds are 6 m/s and 10 m/s. After how many seconds will both students be together again at the starting point for the first time?

600 seconds

300 seconds

450 seconds ✓

— seconds

Each returns to start when: 900/6 = 150 s, and 900/10 = 90 s
LCM(150, 90) = **450 seconds**

Aptitude — Puzzles

Q5 *Aptitude / Puzzles*

In a certain code language, HOUSE is written as JQWUG. Then PICTURE will be written as:

- **RKEVWTG** ✓
- RKEVUTG
- RKEVWVG
- RKEUWTG

> Rule: each letter shifts +2 (H→J, O→Q, U→W, S→U, E→G).
> Applying to PICTURE: P→R, I→K, C→E, T→V, U→W, R→T, E→G = **RKEVWTG**

Q6 *Aptitude / Puzzles*

The 2nd term is analogous to the 1st term in the same way the 4th term is analogous to the 3rd term.
POCKET : PTOECK :: SILVER : ?

- **SRIELV** ✓
- SIRELV
- SRIEVL
- SREILV

> Rule: letter positions rearrange as [1,6,2,5,3,4].
> POCKET (P,O,C,K,E,T) → P,T,O,E,C,K = PTOECK ✓
> Applying to SILVER (S,I,L,V,E,R): positions [1,6,2,5,3,4] = S,R,I,E,L,V = **SRIELV**

Q7 *Aptitude / Puzzles*

The following words get coded in this manner after going through the "star operation":

Words make sentences meaningful ⇒ 8$A, 2!R, 6#C, 14$F
Rain stops every week ⇒ 2^E, 2#O, 8+A, 8!E
Eagerly receives __(a)__ ⇒ 12%R, 4^R, 12+V
Rarely prepares tasty food ⇒ 12%R, 2~S, __(b)__, 8@O

What will come in the blank '(a)'?

- **food** ✓
- rarely
- prepares
- tasty

> Grammatically, "Eagerly receives ___" needs a noun/object to complete the sentence — only **food** works (rarely = adverb, prepares = verb, tasty = adjective needing a noun).

Q8 *Aptitude / Puzzles*

If GIFT is coded as "#!$@" and MUST is coded as "&$*^", then TRUE can be coded as:

- **^$+!** ✓
- &$@%
- &$%~
- ~$+%

> **Note:** this cipher is internally inconsistent (T maps to different symbols in GIFT vs MUST), so it cannot be solved with full certainty. Best practical approach: TRUE shares letters T and U with MUST.
> From MUST (M-U-S-T → &-$-*-^): **U → $**, **T → ^**.
> **^$+!** is the only option containing both required symbols (^ for T, $ for U) — the other three options are missing ^ entirely.

Q9 *Aptitude / Puzzles — Data Analysis*

Consider the text: "*The Ram Mandir is a Hindu temple that is under construction in Ayodhya, Uttar Pradesh, India.*"

Rearrangement algorithm:
– Priority is assigned by word length (more length = higher priority; ties broken by earliest occurrence = higher priority).
– Insert only the words that have even priority into a queue, in order from high to low priority, to get the modified text.

construction Ayodhya Pradesh mandir temple hindu under utter india that The Ram is is in a

a in is is Ram The that india utter under hindu temple mandir Ayodhya construction

in is is that temple mandir construction

construction mandir temple that is is in ✓

Treating "priority" = word length: even-length words are Mandir(6), is(2), temple(6), that(4), is(2), construction(12), in(2) — 7 words.
Ordered by priority high→low (longest first; ties by earliest occurrence):
construction(12) → Mandir(6) → temple(6) → that(4) → is(2,1st) → is(2,2nd) → in(2)
Since it's a queue (FIFO), output order = insertion order → "**construction mandir temple that is is in**"

Data Structures Basics

Q10 *Data Structures Basics*

Which one of the options is the correct difference between array and stack?

Array is an ordered collection of items and stack is not.

Array may contain different data types and stack contains same data types.

Array obeys FIFO rule and stack uses LIFO.

Array is a static object whereas stack is a dynamic object. ✓

Arrays have a fixed size (set at declaration – static). A stack is dynamic – it can grow/shrink at runtime as elements are pushed/popped. (FIFO/LIFO describes queue vs stack, not array vs stack.)

Q11 *Data Structures Basics*

Consider a queue implemented using a single stack:

```
Algorithm
Enqueue(x)
  Push x to stack.

Dequeue
1. If stack is empty then error.
2. If stack has only one element then return it.
3. Recursively pop everything from the stack, store the popped
   item in a variable var, push the var back to stack and
   return var.
```

What will be the time complexity of deleting an element from a stack of size 'n' which will be followed by 'x' insertions?

O(x*n)

O(x)

O(n) ✓

O(1)

Enqueue = simple push ⇒ always O(1). Dequeue (as coded) recursively pops every element down to the base case, then pushes them all back as recursion unwinds — ~2(n−1) operations ⇒ **O(n)** for a single delete. The subsequent x insertions (each O(1)) don't change this – the delete itself remains O(n).

Q12 *Data Structures Basics*

What will be the in-order predecessor of the root node in a complete binary tree with 7 nodes?

Left child of root.

Left child of Left subtree of root.

Right child of Left subtree of root. ✓

Right child of root.

In-order traversal visits left subtree, then root, then right subtree. So the predecessor of the root = the **rightmost node of the left subtree**.

Q13 *Data Structures Basics*

Which of the statement(s) given below is/are true about Bucket sort?
I) It is a sorting technique that generalizes counting sort by partitioning an array into a finite number of buckets.
II) It is best suited to be used on data sets of a limited scope.
III) It uses divide and conquer technique.

All (I), (II) and (III) ✓

Only (I) and (III)

Only (II)

Only (I)

Bucket sort: generalizes counting sort into buckets (I), works best when data is uniformly distributed over a limited range (II), and uses divide-and-conquer – distribute into buckets, sort each, concatenate (III).

Operating System Fundamentals

Q14 *Operating System Fundamentals*

Which of the below given statement(s) is/are correct about priority scheduling algorithm?
I) The algorithm provides a good mechanism to schedule processes properly as defined.
II) Static priority keeps changing till the process completes its execution.
III) Internal priority depends on system limits and memory requirements.

Only (II) and (III)

Only (I) and (III)

Only (II)

Only (III) ✓

I – False. Priority scheduling suffers from **starvation**, so calling it "good... as defined" is misleading.
II – False/contradictory. By definition, **static** priority does NOT change during execution.
III – True. Internal priority is computed using measurable quantities like memory requirements, time limits, I/O-to-CPU ratio, etc.

Q15 *Operating System Fundamentals*

Which of the allocation techniques given in options allocates the smallest hole among the freely available partitions in the main memory?

Best Fit ✓

Next Fit

Worst Fit

First Fit

Best Fit scans all free partitions and allocates the smallest one that is still large enough for the request.

Q16 *Operating System Fundamentals*

An I/O controller belongs to which of the operating system functions given in options?

Process Management

Device Management ✓

Memory Management

File Management

I/O controllers are managed as part of the OS's Device Management function.

C Programming

Q17 *C Programming*

What will be the output of the program given below?

```
#include <stdio.h>
int main()
{
    static int m = 4, n = 2;
    m <<= n | 3 & m;
    n = n >> 2 | m + 1 << n - 1;
    printf("%d  %d", m, n);
    return 0;
}
```

16 34 ✓

0 13

16 12

Statement 1: `m <<= n | 3 & m;` ⇒ RHS = n | (3 & m) since & binds tighter than |.
3 & 4 = 0; 2 | 0 = 2; m <<= 2 ⇒ m = 4 << 2 = **16**.
Statement 2: `n = n >> 2 | m + 1 << n - 1;` (using old n=2)
m+1=17, n-1=1 (computed first, +/- highest precedence); n>>2 = 0; 17<<1 = 34; 0|34 = **34**.
Output: **16 34**

Q18 *C Programming*

What will be the output of the program given below?

```
#include <stdio.h>
int main(void)
{
    int a = 1.00;
    int b = -2.22;
    printf("%.2f  %d  ", --b, --b);
    printf("%.2f  %d", ~a, ~a<<1);
    return 0;
}
```

0.00 -4 0.00 -4 ✓

-3.22 -4 -2.00 -3

-3.22 -4 0.00 -4

Compile time error

Type-mismatch trap: `b` and `a` are **int**, but passed to a `%.2f` specifier (expects double). Int and double use different registers/calling conventions in variadic functions, so the `%.2f` slot reads uninitialized data — typically prints **0.00**.
The `%d` slots correctly read: `--b` twice (b: -2→-3→-4) and `~a<<1` (a=1: ~1=-2, -2<<1=-4). Both show **-4**.
Result: **0.00 -4 0.00 -4**

Q19 *C Programming / Error Finding*

Which of the line(s) generates error(s) in the program given below?

```
#include <stdio.h>
int main()
{
    int array[] = {2, 4, 6, 8, 5, 7};
    int *ar = array + 3;
    +++ar += 1;
    printf("%d ", --(array[1 + 3]));      // Line 1
    printf("%d", (array[!1]) + 2);        // Line 2
    return 0;                              // Line 3
}
```

Line 1, Line 2 and Line 3

No error in the program ✓

Line 2 and Line 3

Line 1

Line 1: `--(array[1+3])` ⇒ `--array[4]` – valid, array[4] is a proper lvalue.
Line 2: `array[!1]` ⇒ `!1` evaluates to 0 first ⇒ `array[0]+2` – valid.
Line 3: standard return statement.
All lines compile and execute correctly.

Q20 *C Programming / Error Finding*

Point out the error if any in the code given below.

```
#include <stdio.h>
int main()
{
    int a = 4;
    int *m, **n;
    m = &a;
    n = &m;                    // Line 1

                               // Line 2
    printf("%d", **n++);       // Line 3
    return 0;
}
```

No error ✓

Line 1 and Line 2

Line 3

Line 2

`n = &m;` is valid (types match: `int**` = address of `int*`). `**n++` parses as `*(*(n++))` due to operator precedence — postfix `++` binds tighter than `*`, so `n` is incremented *after* being dereferenced twice for the print. Valid (if unusual) C; prints `4` correctly.

Q21 *C Programming / Error Finding*

What is the likely cause of the error in the given code below?

```
#include <stdio.h>
int main() {
    int x = 4;
    int b = 5;
    b = x;

    printf("Result: %d", x);
    return 0;
}
```

(Related pointer variant asked in exam:)

```
int *A = NULL;
int b = 5;
A = &b;
printf("%d ", (*A)++);
```

What is the output of the pointer variant shown above?

6

5 ✓

Compile time error

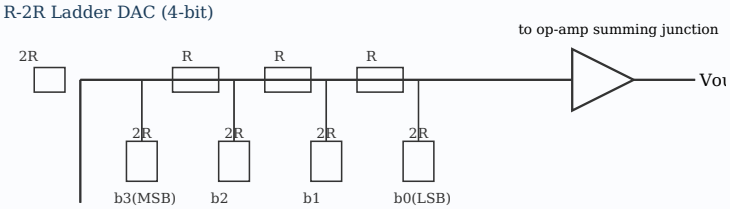
Garbage value

`A = &b` ⇒ A points to b (=5). `(*A)++` is post-increment on the dereferenced value — prints the *current* value (5) first, then increments b to 6.

Digital Circuits & VLSI

Q22 *Digital Circuits*

The reference voltage of a 4-bit R-2R ladder digital-to-analog converter is 5 V. Find the magnitude of analog output voltage for an input of 1010.



3.125 V ✓

2.675 V

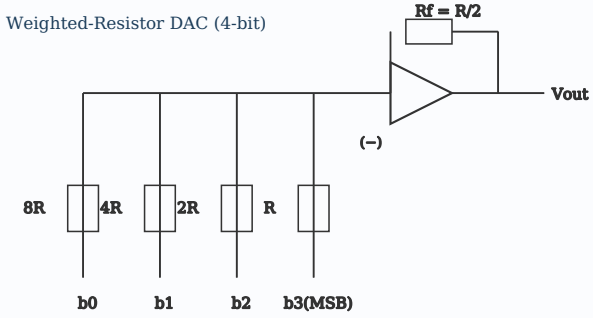
4.115 V

1.565 V

Vout = Vref × (D / 2^n) = 5 × (10 / 16) = **3.125 V**

Q23 *Digital Circuits*

Figure below shows the circuit of a weighted-resistor digital-to-analog converter. VR = −5 V, R = 8 kΩ, Rf = R/2. Find Vout for binary input 0101.



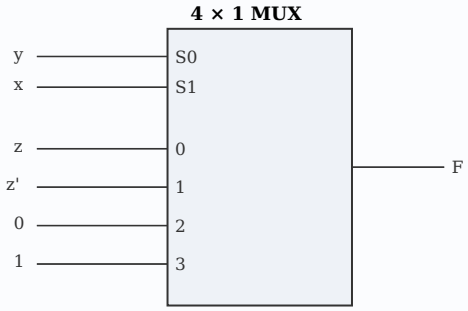
0101 binary input

Vout = 1.5625 V

Bit assignment (MSB→LSB): bit3=0(R), bit2=1(2R), bit1=0(4R), bit0=1(8R)
Vout = −Rf·VR·(bit3/R + bit2/2R + bit1/4R + bit0/8R)
= −(R/2)·VR·(0 + 1/2R + 0 + 1/8R) = −(R/2)·VR·(5/8R) = −5VR/16
= −5(−5)/16 = **1.5625 V**

Q24 *Digital Circuits*

What function is realized by the multiplexer shown?



F(x,y,z) = Σ(1,2,6,7) ✓

F(x,y,z) = Σ(2,4,6,7)

F(x,y,z) = Σ(1,3,6,7)

F(x,y,z) = Σ(1,2,5,7)

S1S0 = xy selects: 00→z, 01→z', 10→0, 11→1
xy=00: F=z → minterm 1 (x'y'z) | xy=01: F=z' → minterm 2 (x'yz')
xy=10: F=0 | xy=11: F=1 → minterms 6,7
F = **Σ(1,2,6,7)**

Q25 *Digital Circuits*

If the output frequency of a T-Flip Flop is 100 Hz, find the input frequency of the T-Flip Flop.

400 Hz

200 Hz ✓

50 Hz

100 Hz

A T flip-flop toggles every clock edge, so output frequency = input / 2. For 100 Hz output ⇒ input = 100 × 2 = **200 Hz**

Q26 *Digital Circuits*

Excess-3 code can be used to perform effectively the addition and subtraction of the following numbers:

Binary

Octal

Hexadecimal

BCD ✓

Excess-3 is a **self-complementing** BCD code (each digit's 9's complement = its own bitwise 1's complement) — this is exactly why it simplifies BCD arithmetic.

Q27 *Digital Circuits*

Under which condition is the circuit "low" when any particular node X is set to logic 1 or 0 through input vector combinations so as to propagate the error from X to the output?

All of the given options

Observability

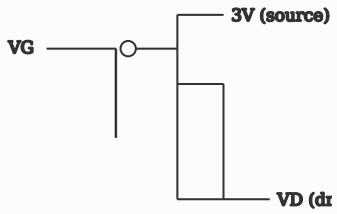
Controllability

Testability ✓

The description combines two things: **setting** a node's value (Controllability) **and propagating** its effect to an observable output (Observability). Testability = the combination of both.

Q28 *Digital Circuits*

The PMOS shown below, with threshold voltage −1 V, will operate in the triode region if:



VD ≥ VG + 1 ✓

VD = VG − 1

VD < VG + 1

VD = VG

For PMOS: VGS = VG−3, Vov = VGS−Vtp = (VG−3)−(−1) = VG−2
Triode condition: VDS > Vov ⇒ (VD−3) > (VG−2) ⇒ **VD > VG+1** (small drain-source drop, VD stays close to source rail – consistent with triode intuition).
Note: the opposite condition, VD < VG+1, corresponds to **saturation**.

Q29 *Digital Circuits*

The logic families of list A can be matched with their best significant characteristic from list B as:

List A	List B
1. MOS	1. High Fan out
2. TTL	2. Saturated bipolar logic
3. ECL	3. Low Power consumption
4. CMOS	4. Higher packing density
5. BiCMOS	5. Low Propagation Delay
	6. Low output Resistance

MOS–4, TTL–2, ECL–5, CMOS–3, BiCMOS–6

MOS → Higher packing density (smaller transistors than bipolar)
TTL → Saturated bipolar logic (classic TTL uses saturated BJTs)
ECL → Low propagation delay (fastest logic family – non-saturated)
CMOS → Low power consumption (hallmark CMOS advantage)
BiCMOS → Low output resistance (bipolar output stage gives strong drive capability)

Q30 *Digital Circuits*

The simplified expression for:
A'B'C'D'E' + A'B'CD'E' + A'BCD'E' + A'BC'D'E' + A'B'C'D'E + A'BCD'E + A'BCDE + AB'C'D'E' + AB'C'D'E + ABCD'E + ABCDE + AB'CDE

A'DE' + B'C'D' + BCE + ACDE

A'D'E' + B'C'D + BCE' + ACDE

A'D'E' + B'C'D' + BCE + ACDE ✓

A'D'E' + B'C'D' + BCE + ACDE'

Minterms = {0,1,4,8,12,13,15,16,17,23,29,31}
A'D'E' covers {0,4,8,12} | **B'C'D'** covers {0,1,16,17}
BCE covers {13,15,29,31} | **ACDE** covers {23,31}
Union covers all 12 given minterms with minimum literals.

Q31 *Digital Circuits*

A counter is designed using JK Flip-Flops. The counter follows this sequence (present-state cycle):

Q2	Q1	Q0	Value
0	1	0	2
1	0	1	5
1	0	0	4
1	1	1	7
1	1	0	6
0	0	1	1
0	0	0	0
0	1	1	3
0	1	0	2 (repeat)

What are the equations for J1 and K1?

J1 = Q0' and K1 = Q0

J1 = Q0 and K1 = Q0'

J1 = Q1' and K1 = Q0'

J1 = Q0' and K1 = Q0' ✓

Using the actual counting sequence (2→5→4→7→6→1→0→3→2…), Q1 **toggles** exactly when Q0(current) = 0, and **holds** when Q0(current) = 1. Using JK excitation rules, this gives **J1 = K1 = Q0'** consistently across all 8 transitions.
(Note: this option may appear as a 4th choice not always visible on-screen – verify against your exam's exact option list.)

Q32 *VLSI*

Ethan is leading a project to develop a high-performance processor for enterprise clients. The processor must deliver top-tier performance and be ready for launch within a year. Budget constraints allow for some flexibility but not unlimited spending. Which design methodology should Ethan choose?

Full Custom Design for maximum performance

Semi-Custom Design for a balance of performance ✓

FPGA Design for rapid prototyping but higher unit costs

Standard Cell Design for faster development

Full Custom = best performance but too slow/expensive for a 1-year timeline with budget limits. Semi-Custom balances strong performance with reasonable cost/time — fits the "flexible but not unlimited" budget and 1-year deadline.

Signals & Systems

Q33 *Signals and Systems*

For the transfer function H(s) = 1 / (s^2 + 5s + 6), ROC: σ > −2, the causality and stability of the system will be:

Casual & unstable

Non-casual & stable

Casual & stable ✓

Non-casual & unstable

Poles at s = −2, −3. ROC (σ > −2) lies to the right of the rightmost pole ⇒ **causal**. ROC includes the jω-axis (σ=0) ⇒ **stable**.

Microprocessors & Microcontroller

Q34 *Microprocessors and Microcontroller*

A program is executed on an 8085 microprocessor with the following instructions. Assume the carry flag was initially clear (CY = 0). What is the content of register C and the status of the carry flag after execution?

```
MVI   B, 3CH        ; Load B with 0011 1100
MVI   A, A5H        ; Load A with 1010 0101
ANA   B             ; Logical AND A with B
RLC                 ; Rotate Accumulator Left through Carry
MOV   C, A          ; Move A to C
```

Content = 48H, Carry = 1

Content = 68H, Carry = 1

Content = 48H, Carry = 0

Content = 48H, Carry = 0

B = 3CH = 0011 1100, A = A5H = 1010 0101
ANA B: A = 1010 0101 AND 0011 1100 = 0010 0100 (24H); **ANA always clears CY ⇒ CY = 0**
RLC: MSB(0) shifts into CY and wraps to LSB ⇒ A = 0100 1000 = **48H**, CY = old MSB = **0**
MOV C,A ⇒ C = 48H